Zadanie 2

Kacper Dusza 131427

Zd.1

```
(.venv) debian@debian:~/Pulpit/UNIVERSITY$ pip install -U nltk
Collecting nltk
  Downloading nltk-3.9.3-py3-none-any.whl.metadata (3.2 kB)
Collecting click (from nltk)
  Downloading click-8.3.1-py3-none-any.whl.metadata (2.6 kB)
Requirement already satisfied: joblib in ./.venv/lib/python3.13/site-packages (from nltk) (1.5.2)
Collecting regex>=2021.8.3 (from nltk)
  Downloading regex-2026.2.28-cp313-cp313-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl.metadata (40 kB)
Collecting tqdm (from nltk)
  Downloading tqdm-4.67.3-py3-none-any.whl.metadata (57 kB)
Downloading nltk-3.9.3-py3-none-any.whl (1.5 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1.5/1.5 MB 20.7 MB/s eta 0:00:00
Downloading regex-2026.2.28-cp313-cp313-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl (802 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 802.1/802.1 kB 64.7 MB/s eta 0:00:00
Downloading click-8.3.1-py3-none-any.whl (108 kB)
Downloading tqdm-4.67.3-py3-none-any.whl (78 kB)
Installing collected packages: tqdm, regex, click, nltk
Successfully installed click-8.3.1 nltk-3.9.3 regex-2026.2.28 tqdm-4.67.3

[notice] A new release of pip is available: 24.3.1 -> 26.0.1
[notice] To update, run: pip install --upgrade pip
```

Zd.2

Podczas ręcznego przetwarzania danych, tekst jest dzielony na części przy użyciu “\n” oraz “\t”. Może zdarzyć się tak że na końcu linii będzie pusty element co spowoduje nierówne wielkości obu części, co będzie prowadziło do błędu

ex2.py × SMSSpamCollection.tsv

```python
import pandas as pd

rawData =open("SMSSpamCollection.tsv").read()
rawData[0:20]

parsedData = rawData.replace(__old: '\t', __new: '\n').split('\n')
parsedData[0:5]

labelList = parsedData[0::2]
textList = parsedData[1::2]

korpus = pd.DataFrame({
    "label": labelList,
    "text": textList
})

print(korpus.head())
```

```
ValueError: All arrays must be of the same length
```

Aby uniknąć takiego błędu warto użyć biblioteki pandas.

```
import pandas as pd

fullCorpus = pd.read_csv(
    filepath_or_buffer: "SMSSpamCollection.tsv", sep='\t', header=None, names=['label', 'text']
)

print(fullCorpus.head())
```

```
/home/debian/Pulpit/UNIVERSITY/.venv/bin/python /home/debian/Pul
  label                                                text
0   ham  I've been searching for the right words to tha...
1  spam  Free entry in 2 a wkly comp to win FA Cup fina...
2   ham  Nah I don't think he goes to usf, he lives aro...
3   ham  Even my brother is not like to speak with me. ...
4   ham                I HAVE A DATE ON SUNDAY WITH WILL!!

Process finished with exit code 0
```

Zd.3

```python
import re

tekst_bazowy = "Kontakt: admin@strona.pl, biuro@firma.com. Cena zestawu to 100.00 PLN. Kod produktu: 12-345."

print("--- 1. re.findall ---")
maile = re.findall( pattern: r'[\w\.-]+@[\w\.-]+', tekst_bazowy)
print(f"Znalezione maile: {maile}")

print("\n--- 2. re.split ---")
czesci = re.split( pattern: r'[ ,.]+', tekst_bazowy)
print(f"Podzielony tekst: {czesci[:5]}...")

print("\n--- 3. re.sub ---")
zamieniony = re.sub( pattern: r'\d',  repl: 'X', tekst_bazowy)
print(f"Po zamianie: {zamieniony}")

print("\n--- 4. re.search ---")
szukaj = re.search( pattern: r'\d+\.\d+', tekst_bazowy)
if szukaj:
    print(f"Pierwsza znaleziona kwota: {szukaj.group()} (pozycja: {szukaj.start()})")

print("\n--- 5. re.match ---")
dopasowanie_poczatku = re.match( pattern: r'Kontakt', tekst_bazowy)
print(f"Czy zaczyna się od 'Kontakt'?: {'Tak' if dopasowanie_poczatku else 'Nie'}")
```

```python
print("\n--- 6. re.fullmatch ---")
kod = "12-345"
czy_kod_poprawny = re.fullmatch( pattern: r'\d{2}-\d{3}', kod)
print(f"Czy '{kod}' to poprawny format kodu?: {'Tak' if czy_kod_poprawny else 'Nie'}")

print("\n--- 7. re.finditer ---")
print("Znalezione liczby i ich lokalizacje:")
for m in re.finditer( pattern: r'\d+', tekst_bazowy):
    print(f" - Liczba: {m.group()}, Start: {m.start()}, Koniec: {m.end()}")

print("\n--- 8. re.escape ---")
problemowy_string = "Czy to kosztuje $100.00?"
bezpieczny_wzorzec = re.escape(problemowy_string)
print(f"Oryginał: {problemowy_string}")
print(f"Wzorzec po re.escape: {bezpieczny_wzorzec}")

czy_znaleziono = re.search(bezpieczny_wzorzec,  string: "Tak, Czy to kosztuje $100.00? Oczywiście.")
print(f"Czy znaleziono dosłowny tekst?: {'Tak' if czy_znaleziono else 'Nie'}")
```

Funkcja re.findall służy do wyodrębnienia wszystkich fragmentów tekstu, które są zgodne z podanym wzorcem.

Dzięki re.split można podzielić ciąg znaków na mniejsze części w każdym miejscu, w którym występuje wzorzec.

Za pomocą re.sub wyszukuje się określone dopasowania i zastępuje je nowym, wskazanym tekstem.

Metoda re.search przeszukuje cały udostępniony tekst, aby odnaleźć i zwrócić pierwsze wystąpienie wzorca.

Funkcja re.match ogranicza sprawdzanie dopasowania wyłącznie do samego początku badanego ciągu znaków.

Użycie re.fullmatch pozwala zweryfikować, czy cały tekst od pierwszej do ostatniej litery odpowiada wzorcowi.

Polecenie re.finditer generuje iterator, który dostarcza obiekty dopasowania wraz z informacją o ich dokładnym położeniu w tekście.

Funkcja re.escape automatycznie zabezpiecza znaki specjalne we wzorcu, aby były one traktowane jako zwykłe znaki tekstowe.

Zd.4

ex4.py

```python
import nltk
import pandas as pd
import string
from nltk import word_tokenize
from nltk.corpus import stopwords

nltk.download('punkt_tab')
nltk.download('stopwords')

def remove_punctuation(text):  1 usage
    return "".join([char for char in text if char not in string.punctuation])

def tokenize(text):  1 usage
    return word_tokenize(text.lower())

def remove_stopwords(tokens):  1 usage
    stop_words = set(stopwords.words('english'))
    return [word for word in tokens if word not in stop_words]

data = pd.read_csv( filepath_or_buffer: 'SPAM text message 20170820 - Data.csv', sep=',')

data['Message_Clean'] = data['Message'].apply(remove_punctuation)
data['Message_Token'] = data['Message_Clean'].apply(tokenize)
data['Message_Stopword'] = data['Message_Token'].apply(remove_stopwords)

data.to_csv( path_or_buf: 'po_zmianach.csv', index=False)

print(f"{data['Message_Clean'].head()}\n\n")
print(f"{data['Message_Token'].head()}\n\n")
print(f"{data['Message_Stopword'].head()}\n\n")
```

```
0    Go until jurong point crazy Available only in ...
1                                Ok lar Joking wif u oni
2    Free entry in 2 a wkly comp to win FA Cup fina...
3          U dun say so early hor U c already then say
4    Nah I dont think he goes to usf he lives aroun...
Name: Message_Clean, dtype: object


0    [go, until, jurong, point, crazy, available, o...
1                        [ok, lar, joking, wif, u, oni]
2    [free, entry, in, 2, a, wkly, comp, to, win, f...
3    [u, dun, say, so, early, hor, u, c, already, t...
4    [nah, i, dont, think, he, goes, to, usf, he, l...
Name: Message_Token, dtype: object


0    [go, jurong, point, crazy, available, bugis, n...
1                        [ok, lar, joking, wif, u, oni]
2    [free, entry, 2, wkly, comp, win, fa, cup, fin...
3        [u, dun, say, early, hor, u, c, already, say]
4    [nah, dont, think, goes, usf, lives, around, t...
Name: Message_Stopword, dtype: object
```